



11

Fetching Data from a Server

The evolution of web technologies has made the interaction of browsers with servers and the processing of server data an integral part of web development. Today, it's challenging to draw a clear line between traditional web pages and full-fledged web applications. At the heart of this evolution is the ability of JavaScript in the browser to make requests to the server, efficiently process the received data, and dynamically display it on the page. This process has become the foundation for creating the interactive and responsive web applications we see today. In this chapter, we will explore various approaches and methods used to fetch data from the server, discuss their impact on the architecture of web applications, and acquaint ourselves with modern practices in this area.

So, in this chapter, we will cover the following topics:

- Working with remote data
- Using the **Fetch API**

- Using **Axios**
- Using **TanStack Query**
- Using **GraphQL**

Technical requirements

You can find the code files for this chapter on GitHub at

<https://github.com/PacktPublishing/React-and-React-Native-5E/tree/main/Chapter11>.

Working with remote data

In the realm of web development, the journey of fetching data from servers has seen remarkable transformations. In the early 90s, the web's infancy with **HTTP 1.0** marked the beginning of server communication. Web pages were static, and HTTP requests were basic, fetching whole pages or static assets. Every request meant establishing a new connection, and interactivity was minimal, mostly limited to HTML forms. Security was also basic, reflecting the nascent state of the web.

The turn of the millennium witnessed a significant shift with the rise of **Asynchronous JavaScript and XML (AJAX)**. This brought an era of enhanced interactivity, allowing web applications to communicate with the server in the background without reloading the whole page. It was powered by the

`XMLHttpRequest` object. Here's a simple example of using `XMLHttpRequest` to fetch data:

```
var xhr = new XMLHttpRequest();
xhr.onreadystatechange = function() {
  if (xhr.readyState == XMLHttpRequest.DONE) {
    if (xhr.status === 200) {
      console.log(xhr.responseText);
    } else {
      console.error('Error fetching data');
    }
  }
};
xhr.open('GET', 'http://example.com', true);
xhr.send();
```

This example illustrates a typical **XHR request**. Success and error responses are managed with callback functions. This reflects the time when asynchronous code relied heavily on callbacks.

As we progressed, HTTP evolved to version **1.1**, enhancing efficiency with persistent connections and standardizing **RESTful APIs**. These APIs used standard HTTP methods and were designed around identifiable resources, greatly improving scalability and developer productivity.

The advent of the **Fetch API** provided a modern, promise-based mechanism to make network requests. Fetch is more powerful and flexible compared to `XMLHttpRequest`. Here's an example of using Fetch:

```
fetch('http://example.com/data')
  .then(response => response.json())
  .then(data => console.log(data))
  .catch(error => console.error('Error:', error));
```

Moreover, there are a lot of tools based on top of the Fetch API and XHR, developed by the community. For example, Axios, GraphQL, and React Query have further simplified server communication and data fetching, enhancing developer experiences.

Axios, a modern HTTP client library, further simplified fetching data with its promise-based API and a host of useful features, such as intercepting requests and responses. Here's how you can use Axios to make a **GET** request:

```
axios.get('http://example.com/data')
  .then(response => console.log(response.data))
  .catch(error => console.error('Error:', error));
```

The example might look identical to the **Fetch** API, but in real projects where you set up interceptors, it becomes a game-changer that saves a lot of time with less code. **Interceptors** allow you to intercept and modify requests before they are sent and responses before they are handled. A common use case is to refresh access tokens when they expire. Interceptors can add the new token to all subsequent requests. By using a library like Axios, a lot of the low-level networking code is abstracted away, allowing you to focus on making requests and handling responses. Interceptors, error handling, and other features help address cross-cutting concerns in a reusable way, leading to cleaner code.

Next is **GraphQL**, which revolutionized data fetching by allowing clients to request exactly the data they need, eliminating over-fetching and under-fetching issues. It offers a flexible and efficient way to retrieve data from servers. Instead of predefined endpoints, clients specify their data requirements, and servers respond with precisely the requested data. This reduces network load and enhances application performance.

```
import { GraphQLClient, gql } from 'graphql-request';
const endpoint = 'http://example.com/graphql';
const client = new GraphQLClient(endpoint);
const query = gql`
  query {
    user(id: 123) {
```

```
      name
      email
    }
  }
};
client.request(query)
  .then(data => console.log(data))
  .catch(error => console.error('Error:', error));
```

Here, we request the user by `ID`, specifying only two fields: `name` and `email`. Regardless of the user object's size, the GraphQL server efficiently handles it, sending only the requested data to the client.

One more tool I would like to explore is **React Query**. The library is designed to simplify data fetching and state management in React applications. It abstracts away the complexities of fetching and caching data, handles background updates, and provides Hooks for easy integration with components. React Query enhances the development process by making it straightforward to work with server data in a highly efficient and maintainable manner.

```
import { useQuery } from 'react-query';
function UserProfile({ userId }) {
  const { data, error, isLoading } = useQuery(userId, fetchUser);
  if (isLoading) return <div>Loading...</div>;
```

```
if (error) return <div>Error: {error.message}</div>;
return (
  <div>
    <h1>{data.name}</h1>
    <p>Email: {data.email}</p>
  </div>
);
}
```

As you can see, we don't even need to handle errors or set up and update the loading state manually. Everything is provided by one Hook.

Another remarkable development in server communication is **WebSockets**, enabling real-time, bidirectional communication. This is a game-changer for applications requiring live data updates, like chat apps or trading platforms. Below is a basic example of using WebSockets:

```
const socket = new WebSocket('ws://example.com');
socket.onopen = function(event) {
  console.log('Connection established');
};
socket.onmessage = function(event) {
  console.log('Message from server ', event.data);
};
socket.onerror = function(error) {
```

```
console.error('WebSocket Error ', error);  
};
```

Here, we are still using the callback approach due to the mental model of bidirectional communication.

In conclusion, the evolution of server communication in web development has been pivotal in enhancing user experiences and developer productivity. From the rudimentary stages of **HTTP 1.0** to the sophisticated tools of today, we have witnessed a significant transformation. The introduction of technologies like Ajax, the Fetch API, Axios, GraphQL, and React Query not only streamlined server interactions but also standardized asynchronous behavior in applications. These advancements have been critical in efficiently managing states like loading, errors, and offline scenarios. The integration of these tools in modern web applications signifies a leap forward in building more responsive, robust, and user-friendly interfaces. It's a testament to the ever-evolving nature of technology and its profound impact on both the creation and consumption of web content.

In the next section, we will explore real examples of how to fetch data from a server using the Fetch API.

Using the Fetch API

Let's explore how we can retrieve data from a server in practice. We'll start with the **Fetch API**, the most common and fundamental approach provided by web browsers.

Before we begin, let's create a small application that fetches user data from GitHub and displays their avatar and basic information on the screen. To do this, we'll need an empty **Vite** project with React. You can create it with the following command:

```
npm create vite@latest
```

Since we're using **TypeScript** in our examples, let's start by defining the `GitHubUser` interface and all the necessary parameters.

To find out what data the server returns, we often need to refer to the documentation, usually provided by backend developers. In our case, since we're using the GitHub REST API, we can find user information in the official GitHub documentation at this link: <https://docs.github.com/en/rest/users/users?apiVersion=2022-11-28>.

Let's create the `GitHubUser` interface as follows:

```
export interface GitHubUser {  
  login: string;  
  id: number;  
  avatar_url: string;  
  html_url: string;  
  gists_url: string;  
  repos_url: string;  
  name: string;  
  company: string | null;  
  location: string | null;  
  bio: string | null;  
  public_repos: number;  
  public_gists: number;  
  followers: number;  
  following: number;  
}
```

These are the essential fields we will use in our application. In reality, there are more fields in the `user` object, but I've included the ones we'll use.

Now that we know the fields the user will have, let's create a component that will display the user data on the screen:

```
const UserInfo = ({ user }: GitHubUserProps) => {  
  return (  
    <div>  
      <img src={user.avatar_url} alt={user.login} width="100" height="100" />  
    </div>  
  );  
}
```

```

    <h2>{user.name || user.login}</h2>
    <p>{user.bio}</p>
    <p>Location: {user.location || "Not specified"}</p>
    <p>Company: {user.company || "Not specified"}</p>
    <p>Followers: {user.followers}</p>
    <p>Following: {user.following}</p>
    <p>Public Repos: {user.public_repos}</p>
    <p>Public Gists: {user.public_gists}</p>
    <p>
      GitHub Profile:{" "}
      <a href={user.html_url} target="_blank" rel="noopener noreferrer">
        {user.login}
      </a>
    </p>
  </div>
);
};

```

Here, we render their avatar and some useful information about the user together with a link to open their GitHub profile page.

Let's now take a look at the `App` component, where we handle the server data retrieval logic:

```

function App() {
  const [user, setUser] = useState<GitHubUser>();
  const [loading, setLoading] = useState(true);
  useEffect(() => {

```

```
    setLoading(true);
    fetch("https://api.github.com/users/sakhnyuk")
      .then((response) => response.json())
      .then((data) => setUser(data))
      .catch((error) => console.log(error))
      .finally(() => setLoading(false));
  }, []);
```

We use the `useState` hook to store `user` data and loading state. In the `useEffect`, we make a Fetch API request to fetch data from the GitHub API. As you can see, the `fetch` function takes a URL as an argument. We process the response, save it in the state, handle errors with the `catch` block, and finally, turn off the loading process with the `finally` block.

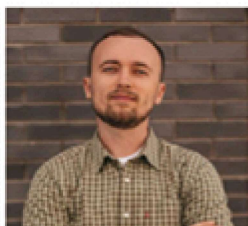
To complete the application, we display the retrieved user data:

```
return (
  <div>
    {loading && <p>Loading...</p>}
    {!loading && !user && <p>No user found.</p>}
    {user && <UserInfo user={user} />}
  </div>
);
```

You can run your application using the following command:

```
npm run dev
```

Open the link that will appear in the terminal and you'll see:



Michael Sakhniuk

Software Engineer at KappaPay (kappapay.com)

Location: Barcelona, Spain

Company: [@kappapay](#)

Followers: 67

Following: 22

Public Repos: 15

Public Gists: 2

GitHub Profile: [sakhnyuk](#)

Figure 11.1: GitHub user requested by the Fetch API

Now you know how to fetch data using the Fetch API. Let's explore the implementation of a similar application where we request data using other tools.

Using Axios

In this section, we will explore one of the most popular libraries for working with the server, called **Axios**. This library is similar to the Fetch API but also provides additional features that make it a powerful tool for handling requests.

Let's take our previous project and make some changes to it. First, let's install Axios as a dependency:

```
npm install axios
```

One of Axios's features is the ability to create instances with specific configurations, such as headers, base URLs, interceptors, and more. This allows us to have a preconfigured instance tailored to our needs, reducing code repetition and making it more scalable.

Let's create an API class that encapsulates all the necessary logic for working with the server:

```
class API {
  private apiInstance: AxiosInstance;
  constructor() {
    this.apiInstance = axios.create({
      baseURL: "https://api.github.com",
    });
    this.apiInstance.interceptors.request.use((config) => {
      console.log("Request:", `${config.method?.toUpperCase()} ${config.url}`);
      return config;
    });
    this.apiInstance.interceptors.response.use(
      (response) => {
        console.log("Response:", response.data);
        return response;
      },
      (error) => {
        console.log("Error:", error);
        return Promise.reject(error);
      }
    );
  }
  getProfile(username: string) {
    return this.apiInstance.get<GitHubUser>(`/users/${username}`);
  }
}
export default new API();
```

In the constructor of this class, we create and store an Axios instance and set the base URL, eliminating the need to repeat this

domain in future requests. Next, we configure interceptors for each request and response. This is done for demonstration purposes, so when we run the application, we can see all the requests and responses in the console logs:

```
Request: GET /users/sakhnyuk api.ts:13  
Response: api.ts:19  
{login: 'sakhnyuk', id: 32235469, node_id: 'MDQ6VXNlc  
▶ jMyMjMINDY5', avatar_url: 'https://avatars.githubusercontent.com/u/32235469?v=4', gravatar_id: '', ...}
```

Figure 11.2: Axios interceptor logs

Now, let's see how the `App` component, which uses our new API class, will look:

```
function App() {  
  const [user, setUser] = useState<GitHubUser>();  
  const [loading, setLoading] = useState(true);  
  useEffect(() => {  
    setLoading(true);  
    api  
      .getProfile("sakhnyuk")  
      .then((res) => setUser(res.data))  
      .finally(() => setLoading(false));  
  }, []);  
  return (  
    <div>
```



```
    {loading && <p>Loading...</p>}  
    {!loading && !user && <p>No user found.</p>}  
    {user && <UserInfo user={user} />}  
  </div>  
);  
}
```

As mentioned earlier, Axios is not significantly different from the Fetch API, but it offers more powerful functionality, making it easy to create more complex solutions for working with server data.

In the next section, we will explore the same application implemented using TanStack Query.

Using TanStack Query

TanStack Query, more commonly known as **React Query**, is a library that has taken server interaction to a new level. This library allows us to request data and cache it. As a result, we can call the same `useQuery` hook a lot of times during one rendering, but only one request will be sent to the server. The library also includes built-in loading and error states, simplifying the handling of request states.

To get started, let's install the library as a dependency for our project:

```
npm install @tanstack/react-query
```

Next, we need to configure the library by adding the `QueryClientProvider`:

```
const queryClient = new QueryClient();
ReactDOM.createRoot(document.getElementById("root")).render(
  <QueryClientProvider client={queryClient}>
    <App />
  </QueryClientProvider>
);
```

After this setup, we can start working on the app. One of the unique features of this library is that it is agnostic to the tool you use for data fetching. You just need to provide a promise function that returns the data. Let's create such a function using the Fetch API:

```
const userFetcher = (username: string) =>
  fetch("https://api.github.com/users/sakhnyuk")
    .then((response) => response.json());
```

Now, let's take a look at how simple our `App` component has become:

```
function App() {  
  const {  
    data: user,  
    isPending,  
    isError,  
  } = useQuery({  
    queryKey: ["githubUser"],  
    queryFn: () => userFetcher("sakhnyuk"),  
  });  
  return (  
    <div>  
      {isPending && <p>Loading...</p>}  
      {isError && <p>Error fetching data</p>}  
      {user && <UserInfo user={user} />}  
    </div>  
  );  
}
```

Now, all the logic for making the request and handling loading and error states is contained within a single `useQuery` hook.

In the next section, we will explore an even more powerful tool for data fetching using GraphQL.

Using GraphQL

Earlier in this chapter, we discussed what **GraphQL** is and how it allows us to specify the exact data we want from the

server, reducing the amount of transferred data and speeding up data fetching.

In this example, we will explore GraphQL in conjunction with the `@apollo/client` library, which provides similar functionality to React Query but works with GraphQL queries.

To begin, let's install the necessary dependencies using the following command:

```
npm install @apollo/client graphql
```

Next, we need to add a provider to our application:

```
const client = new ApolloClient({
  uri: "https://api.github.com/graphql",
  cache: new InMemoryCache(),
  headers: {
    Authorization: 'Bearer YOUR_PAT', // Put your GitHub personal access token here
  },
});
ReactDOM.createRoot(document.getElementById("root")).render(
  <ApolloProvider client={client}>
    <App />
  </ApolloProvider>
);
```

At this stage, during the client setup, we specify the server URL we want to work with, caching settings, and authentication. In earlier examples, we used the public GitHub API, but GitHub also supports GraphQL. For this, we need to provide a **GitHub personal access token**, which you can obtain in your GitHub profile settings.

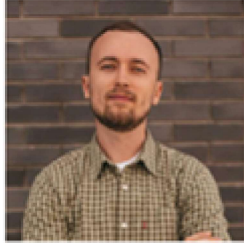
For our example, to demonstrate how we can select only the necessary fields we need, let's trim down the user data. Here's how our GraphQL query in the component will look:

```
const GET_GITHUB_USER = gql`
  query GetGithubUser($username: String!) {
    user(login: $username) {
      login
      id
      avatarUrl
      bio
      name
      company
      location
    }
  }
`;
```

Now that everything is set up, let's see what the `App` component will look like:

```
function App() {  
  const { data, loading, error } = useQuery(GET_GITHUB_USER, {  
    variables: { username: "sakhnyuk" },  
  });  
  if (loading) return <p>Loading...</p>;  
  if (error) return <p>Error fetching data</p>;  
  const user = data.user;  
  return (  
    <div>  
      <UserInfo user={user} />  
    </div>  
  );  
}
```

Similar to React Query, we have access to the loading state, errors, and the actual data. When we open the application, we will see the result:



Michael Sakhniuk

Software Engineer at KappaPay (kappapay.com)

Location: Barcelona, Spain

Company: @kappapay

Figure 11.3: GitHub user requested by GraphQL

To ensure that the server returns exactly the data we requested, we can open Chrome Dev Tools, go to the **Network** tab, and inspect our request:

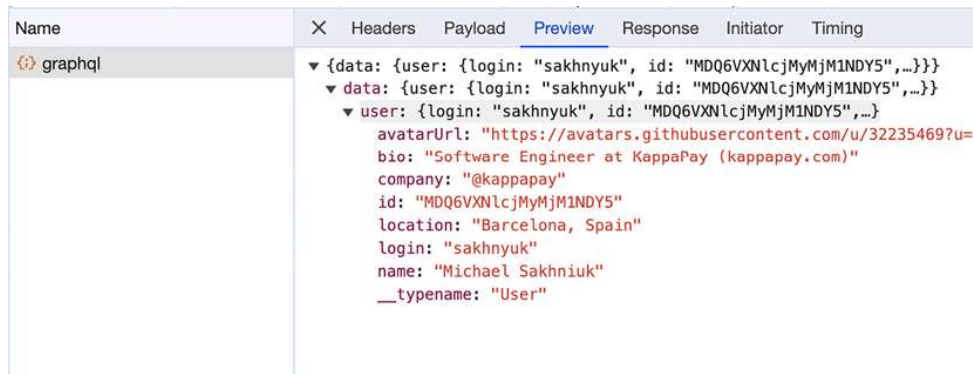


Figure 11.4: GraphQL request

As shown in *Figure 11.4*, the server sent us precisely the data we specified in the query. You can experiment with query parameters to see the difference.

Summary

In this chapter, we explored how to fetch data from the server. We began by briefly reviewing the history of client-server communication and highlighting the primary methods of interacting with servers. Next, we built an application to retrieve GitHub user data using the Fetch API, Axios, TanStack Query, and Apollo GraphQL.

The techniques you learned in this chapter will enable you to significantly expand the capabilities of your own web applications. By efficiently fetching data from the server, you can create dynamic, data-driven experiences for your users. Whether

you are building a social media app that displays real-time feeds, an e-commerce site with up-to-date product information, or a dashboard that visualizes live data, the skills you gained will prove invaluable.

In the next chapter, we will delve into managing the application state using state management libraries.